📘 MODULE 14: ADVANCED SPL

Format per command: **Purpose → Syntax → Example → Real use → Mistake → Performance.**

eventstats

- **Purpose:** compute a statistic and **add it to every event** (events are kept).
- **Syntax:** | eventstats <func>(<field>) AS <name> [by <field>]
- **Example:**

spl

```
index=_internal sourcetype=splunkd_access
| eventstats avg(bytes) AS avg_bytes by clientip
| where bytes > avg_bytes*3
```

- **Real use:** find events far above the average for that user/host.
- **Mistake:** expecting it to collapse rows (that's stats).
- **Performance:** holds all events in memory, so filter first.

streamstats

- **Purpose: running** statistics, calculated event by event in order.
- **Syntax:** | streamstats [window=N | time_window=5m] <func>(<field>) AS <name> [by <field>]
- **Example (burst detection):**

spl

```
index=_internal sourcetype=splunkd_access
| sort 0 _time
| streamstats time_window=5m count AS requests_5m by clientip
| where requests_5m>50
```

- **Real use:** "5+ failures within 5 minutes" detections, running totals.
- **Mistake:** forgetting sort 0 _time. Events arrive newest-first, so the window is wrong.
- **Performance:** it's sequential, and by fields help limit work.

mvexpand

- **Purpose:** turn one event with a multivalue field into **one event per value**.
- **Syntax:** | mvexpand <field> [limit=N]
- **Example:** | makeresults | eval ips=split("1.1.1.1,2.2.2.2",",") | mvexpand ips
- **Real use:** a field holds many IPs/URLs and you want to count each.
- **Mistake:** expanding huge multivalue fields (memory limits).
- **Performance:** multiplies event count. Use fields first to drop unneeded fields.

makemv

- **Purpose:** split a single string field into a multivalue field.
- **Syntax:** | makemv delim="," <field>
- **Example:** | makeresults | eval tags="a,b,c" | makemv delim="," tags
- **Mistake:** delimiter mismatch (spaces, quotes).
- **Alternative:** eval x=split(tags,",").

Multivalue functions (used in eval)

Function	Purpose	Example	Result
split(str,delim)	String to multivalue	split("a,b,c",",")	a, b, c
mvjoin(mv,delim)	Multivalue to string	mvjoin(tags,"; ")	a; b; c
mvcount(mv)	Number of values	mvcount(tags)	3
mvindex(mv,n)	Value at position (0-based; negative counts from the end)	mvindex(tags,0) / mvindex(tags,-1)	a / c

Real use: values(user) after stats is multivalue, so mvcount(user)>10 flags IPs touching many users.
Mistake: mvindex is 0-based, not 1-based.

coalesce and fillnull

- coalesce(a,b,c): first **non-null** value. **Real use:** merge user, src_user, Account_Name from different sourcetypes into one field.
- fillnull value="N/A" [fields]: replace nulls. **Mistake:** without value=, it fills 0. **Performance:** cheap.
- **Why it matters:** stats ... by field **drops events where the field is null**. fillnull first keeps them.

convert

- **Purpose:** convert field types and time formats.
- **Example:** | convert timeformat="%F %T" ctime(_time) AS readable
- **Also:** num(field) converts to number.
- **Mistake:** forgetting that ctime needs an epoch number. Often eval strftime() is clearer.

foreach

- **Purpose:** apply the same eval to many fields.
- **Syntax:** | foreach <wildcard-fields> [eval <<FIELD>> = ...]
- **Example:** | foreach count_* [eval <<FIELD>> = round('<<FIELD>>',1)]
- **Mistake:** on the right side of =, wrap <<FIELD>> in **single quotes** so it is read as a field name.
- **Real use:** clean or scale many columns after chart ... by.

map

- **Purpose:** run a **separate search for each input row**.
- **Syntax:** | map search="search index=main src_ip=src_ip" maxsearches=10
- **Example:**

spl

```
index=_internal sourcetype=splunkd_access | top limit=3 clientip
| map search="search index=_internal clientip=$clientip$ | head 2" maxsearches=3
```

- **Real use:** follow-up investigation per suspicious IP.
- **Mistake:** large inputs (default maxsearches=10).
- **Performance: very expensive** (one full search per row). Last resort.

rangemap

- **Purpose:** label numeric values into named ranges.
- **Syntax:** | rangemap field=count low=0-10 elevated=11-50 default=severe
- **Result:** new field range = low/elevated/severe.
- **Alternative:** eval case() (more flexible, and you already know it).

fieldsummary

- **Purpose: summarize every field** in the results (count, distinct count, sample values).
- **Example:** index=main sourcetype=linux_secure | fieldsummary maxvals=5
- **Real use:** quickly understand a new data source during onboarding.
- **Performance:** run on a small sample (| head 1000).

metadata

- **Purpose: very fast** listing of hosts/sources/sourcetypes with first/last seen times.
- **Syntax:** | metadata type=hosts|sources|sourcetypes [index=<idx>]
- **Example:**

spl

```
| metadata type=hosts index=*
| eval last_seen=strftime(recentTime,"%F %T")
| table host totalCount last_seen
```

- **Real use:** “which hosts are sending data and when did they last send?”
- **Mistake:** it needs the leading pipe and doesn’t accept filters on event content. Times are epoch, so format them.
- **Performance:** excellent, because it reads metadata only.

rest

- **Purpose:** query Splunk’s own **REST API** from a search, for configuration and status information.
- **Example:**

spl

```
| rest /services/data/indexes splunk_server=local
| table title currentDBSizeMB maxTotalDataSizeMB totalEventCount
```

- **Real use:** admins list indexes and sizes, saved searches, users, apps, and licenses.
- **Mistake:** missing leading pipe, or your role lacks permission to those endpoints. Output fields vary by endpoint.

- **Performance:** cheap, but available fields depend on the endpoint. Use | rest ... | fieldsummary to discover them.

14.1 Hands-on

1. With eventstats, flag requests where bytes is more than 3x that client's average.
2. Use streamstats time_window=5m to find any IP with more than 20 requests in 5 minutes.
3. Create tags="a,b,c" with makeresults, then use makemv, mvcount, mvindex, and mvexpand.
4. Use metadata type=hosts to list hosts and when each last sent data.
5. Use rest to list your indexes and their sizes.

14.2 Interview Q&A

- **stats vs eventstats vs streamstats?** stats collapses, eventstats adds the aggregate to every event, streamstats computes running values in order.
- **How do you detect "5 failures in 5 minutes"?** Sort by _time, use streamstats time_window=5m count by src_ip, then where.
- **When use mvexpand?** To process each value of a multivalue field separately.
- **Why is map discouraged?** It runs one search per row.
- **metadata vs stats?** metadata is much faster, but only provides host/source/sourcetype summaries.

14.3 Revision

eventstats = add to each event · streamstats = running (sort first) · mvexpand = one event per value · mvindex is 0-based · fillnull before stats by · avoid map · metadata/rest are generating commands.